ABC Insurance -
Snowflake Data Engineering Project

Team Members

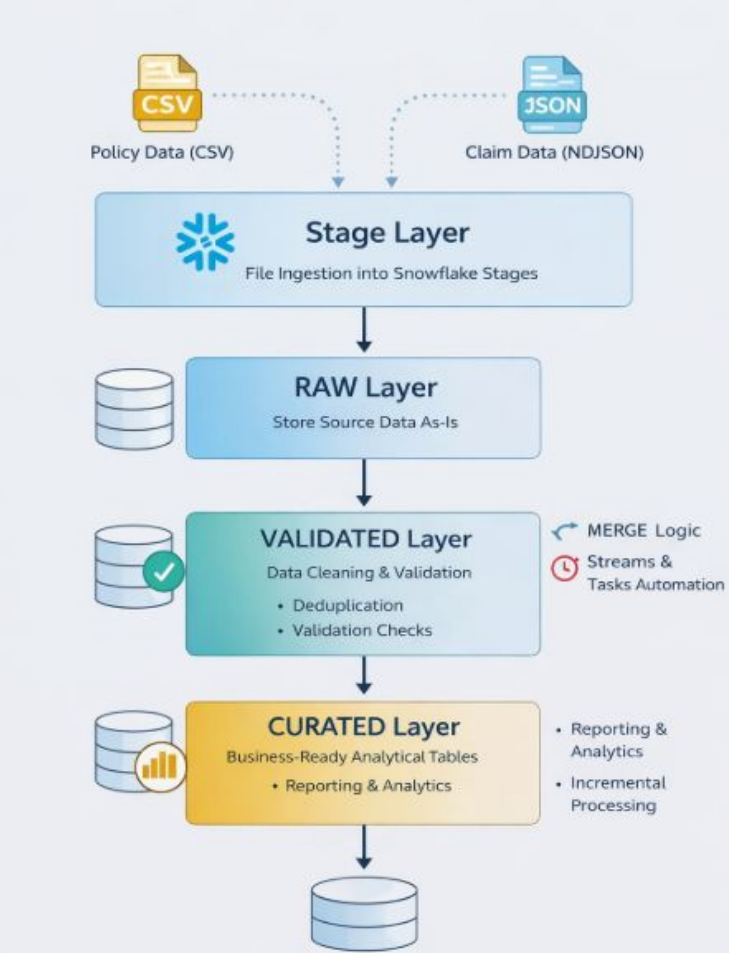
- Suhani Gahukar
- Khushi Dekate
- Mayuri Motwani

Project Objective

- Establish a trusted and consistent view of policy and claims data.
- Improve data quality, completeness, and validation across insurance records.
- Enable better decision-making for underwriting, claims, and distribution teams.
- Standardize business metrics for customer behavior and claim severity analysis.
- Provide insight into operational gaps and growth opportunities.
- Build a scalable Snowflake-based analytics platform for reporting and KPIs.

Architecture Overview

- Stage Layer: Ingest policy and claim files into Snowflake internal stages.
- RAW Layer: Store source data exactly as received without transformation.
- VALIDATED Layer: Perform data cleaning, deduplication, and validation checks.
- CURATED Layer: Create business-ready tables for reporting and analytics.
- Implement incremental processing using MERGE logic.
- Automate pipeline using Streams and Tasks for efficient data processing.



Data Sources & File Formats

- Policy data received as CSV files (structured data).
- Claim data received as NDJSON files (semi-structured data).
- Master files contain the initial full dataset.
- Incremental files contain newly added or updated records.
- CSV data loaded into structured Snowflake tables.
- JSON data stored using VARIANT data type for flexible schema handling.

RAW Layer Processing

- Load policy CSV files into structured tables using COPY INTO command.
- Load claims NDJSON files into tables using the VARIANT data type for semi-structured data.
- Store data exactly as received from the source without any transformation.
- Capture file metadata and load timestamps for auditing and tracking.
- Maintain separate RAW tables for policies and claims data.
- Enable traceability and recovery by preserving the original source data.

Semi-Structured Data Handling

- Use VARIANT data type to store JSON claim data in Snowflake.
- Handle NDJSON files where each record is stored as a JSON object.
- Extract nested fields using dot notation (e.g., data:customer_id).
- Support arrays and nested structures within JSON records.
- Enable schema-on-read, allowing flexible structure for incoming data.
- Convert required JSON fields into structured columns during validation stage.

VALIDATED Layer Processing

- Perform data cleaning and transformation on RAW layer data.
- Remove duplicate records using primary keys like policy number or claim ID.
- Apply data quality checks such as null validation, date validation, and premium validation.
- Extract structured fields from JSON VARIANT columns.
- Standardize data formats (dates, text, phone numbers, etc.).
- Ensure referential integrity between policies and claims tables.

INCREMENTAL Processing

- Process new and updated records using incremental data files.
- Implement MERGE logic to update existing records and insert new records.
- Prevent duplicate data loading during repeated pipeline runs.
- Ensure idempotent processing, allowing safe re-execution of pipelines.
- Improve pipeline efficiency by avoiding full data reloads.

Security & Governance

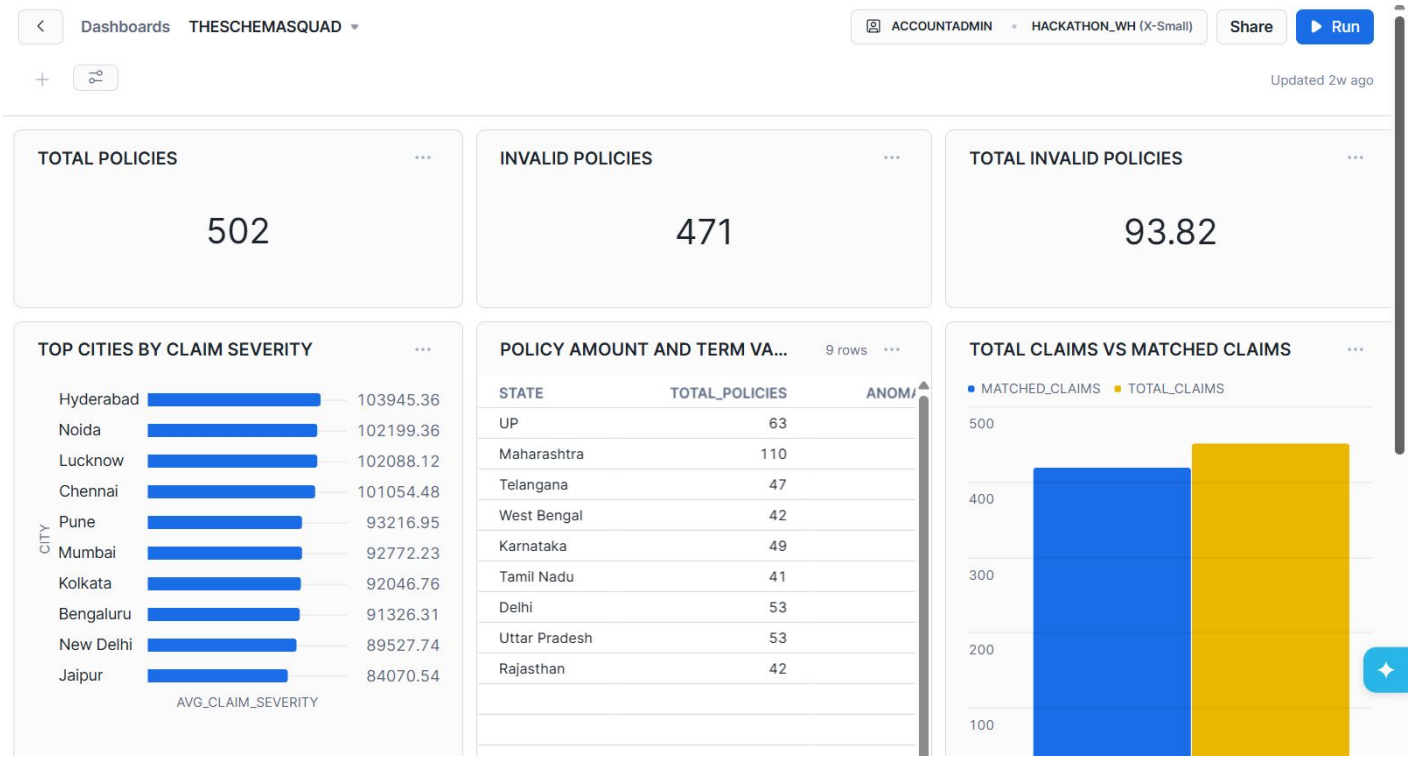
- Implement Dynamic Data Masking to protect sensitive fields like SSN and contact details.
- Apply Row Access Policies to restrict access based on region or role.
- Use RBAC (Role-Based Access Control) to manage user permissions.
- Maintain audit logs and metadata tracking for monitoring data access.
- Ensure data privacy and compliance with security policies.

Automation with Streams & Tasks

- Use Streams to capture changes (inserts, updates, deletes) in tables.
- Schedule Tasks to automatically process incremental data.
- Enable continuous data pipeline execution without manual intervention.
- Reduce processing time by handling only changed data.
- Improve overall data pipeline efficiency and reliability.

Project Outcome

- Successfully processed structured (CSV) and semi-structured (JSON) data.
- Built a scalable Snowflake data architecture with multiple layers.
- Ensured data security through masking, RBAC, and access policies.
- Automated the pipeline using Streams and Tasks.
- Generated business-ready datasets for KPI reporting and analytics.



THANK YOU !